

Smart Traffic Signal Control using Reinforcement Learning

Oct 6, 2026 · @zehra nisar

Zehra Nisar · University Institute of Information Technology · CAI-461 Reinforcement Learning (BSCS-7, Fall 2026)

Abstract

A deep-RL agent (DQN) cut the average queue at a simulated intersection by 80% in normal traffic and 58% in rush traffic compared with a 30-second fixed-timer signal, but a simple longest-queue rule was still best in low traffic.

Fixed-timer signals cannot react when the busy direction changes during the day. This project builds a four-way intersection simulator whose heavy direction swaps halfway through each simulated hour, and trains three agents on it: tabular Q-learning, DQN and PPO. They are compared with a random controller, two fixed timers (30 s and 45 s) and a longest-queue-first rule on 30 unseen traffic scenarios per level (low, normal, rush).

In normal traffic DQN averaged 5.1 waiting cars against 25.2 for the 30-second fixed timer; in rush traffic 31.9 against 75.6. In low traffic the longest-queue rule was best (1.9 cars; DQN 2.1). PPO was the weakest of the three RL agents. A web demo shows any baseline and any RL agent side by side on the same traffic.

Introduction

A fixed-timer signal gives each direction the same green time whether the road is empty or full, so when traffic shifts toward one direction cars pile up on the busy side while the other side sits green with few cars.

This project asks whether a reinforcement learning (RL) agent can learn to choose which direction gets green, using only the current queue lengths, and whether it beats simpler controllers. It makes three contributions:

- a simulated four-way intersection (a Gymnasium environment) in which the busy direction changes halfway through each hour;
- three RL agents (tabular Q-learning, DQN, PPO) compared with four non-learning controllers on the same unseen traffic;
- a web demo that runs any baseline and any RL agent side by side.

Simulation environment

The environment models one four-way intersection with four incoming lanes (North, South, East, West) and two signal phases: North–South green or East–West green. Every 5 seconds the agent picks the phase for the next 5 seconds. One episode is one simulated hour (720 decisions).

Element	Definition
State (7 values)	queue length of each lane (divided by 60), current phase (one-hot), time in current phase (divided by 60, capped at 1)
Action	keep the current phase or switch to the other one
Reward	–(waiting cars, summed over every second of the 5-second step) ÷ 100, minus 0.3 for each phase switch
Switching cost	3 seconds of yellow, during which no car passes
Discharge	a green lane releases up to 0.5 cars per second
Queue limit	60 cars per lane; arrivals beyond that are dropped

Cars arrive at random: each second a lane receives one car with probability equal to its arrival rate. The base rate is 0.08 cars/s per lane (low), 0.14 (normal) or 0.20 (rush), multiplied in every scenario by a random factor between 0.6 and 1.4 per lane.

In the first half-hour one direction pair is the busy one (its rates $\times 1.4$, the other pair's $\times 0.6$); in the second half the roles swap. This shifting demand is what a fixed timer cannot follow.

Methods

Baselines (no learning)

Controller	Rule
Random	picks a phase at random every 5 seconds
Fixed timer (30 s)	switches phase every 30 seconds
Fixed timer (45 s)	switches phase every 45 seconds
Longest queue first	after at least 10 seconds of green, gives green to the direction with more waiting cars

RL agents

Agent	Input	Settings
Q-learning (table)	total queue of each direction pair in 8 bins, phase, time in phase in 4 bins (512 states, 1,024 values)	learning rate 0.1, discount 0.95, exploration 1.0 $\rightarrow$ 0.05 over the first 60% of 3,000 episodes
DQN (Stable-Baselines3)	the 7-value state	two hidden layers of 64, learning rate $5e-4$, replay buffer

Agent	Input	Settings
		100,000, batch 128, discount 0.95, target update every 2,000 steps, exploration over 40% of training, 300,000 steps
PPO (Stable-Baselines3)	the 7-value state	two hidden layers of 64, learning rate 3e-4, 4 parallel environments, 1,440 steps per update, batch 360, discount 0.95, entropy coefficient 0.01, 300,000 steps

Each training episode uses a randomly chosen traffic level (Q-learning cycles through low, normal and rush), so one agent per algorithm handles all three levels.

Experimental setup

Every controller is tested on the same 30 traffic scenarios per level (low, normal, rush), generated from seeds 5000–5029 that were not used in training. A scenario fixes the random lane factors, the busy direction and every car arrival, so all controllers face exactly the same traffic.

Two measures are computed per scenario and then averaged:

- **Average queue length:** cars waiting at the intersection, averaged over every second of the hour. Lower is better.
- **Cars passed:** cars that cleared the intersection during the hour. Higher is better.

Tables give the mean $\pm$ standard deviation over the 30 scenarios. Each RL agent was trained once (a single random seed).

Results

DQN has the lowest average queue in normal (5.1 cars) and rush (31.9) traffic; in low traffic longest-queue-first is best (1.9, with DQN at 2.1).

Table 1. Average queue length (cars), mean $\pm$ std over 30 unseen scenarios. Lower is better.

Controller	Low	Normal	Rush
Random	7.7 $\pm$ 4.7	59.4 $\pm$ 21.9	115.8 $\pm$ 17.8
Fixed timer (30 s)	3.9 $\pm$ 0.6	25.2 $\pm$ 15.7	75.6 $\pm$ 20.2
Fixed timer (45 s)	5.3 $\pm$ 0.8	21.3 $\pm$ 11.3	67.6 $\pm$ 21.0
Longest queue first	1.9 $\pm$ 0.3	8.0 $\pm$ 6.9	95.0 $\pm$ 45.9
Q-learning (RL)	2.4 $\pm$ 0.3	5.6 $\pm$ 1.6	42.5 $\pm$ 31.5

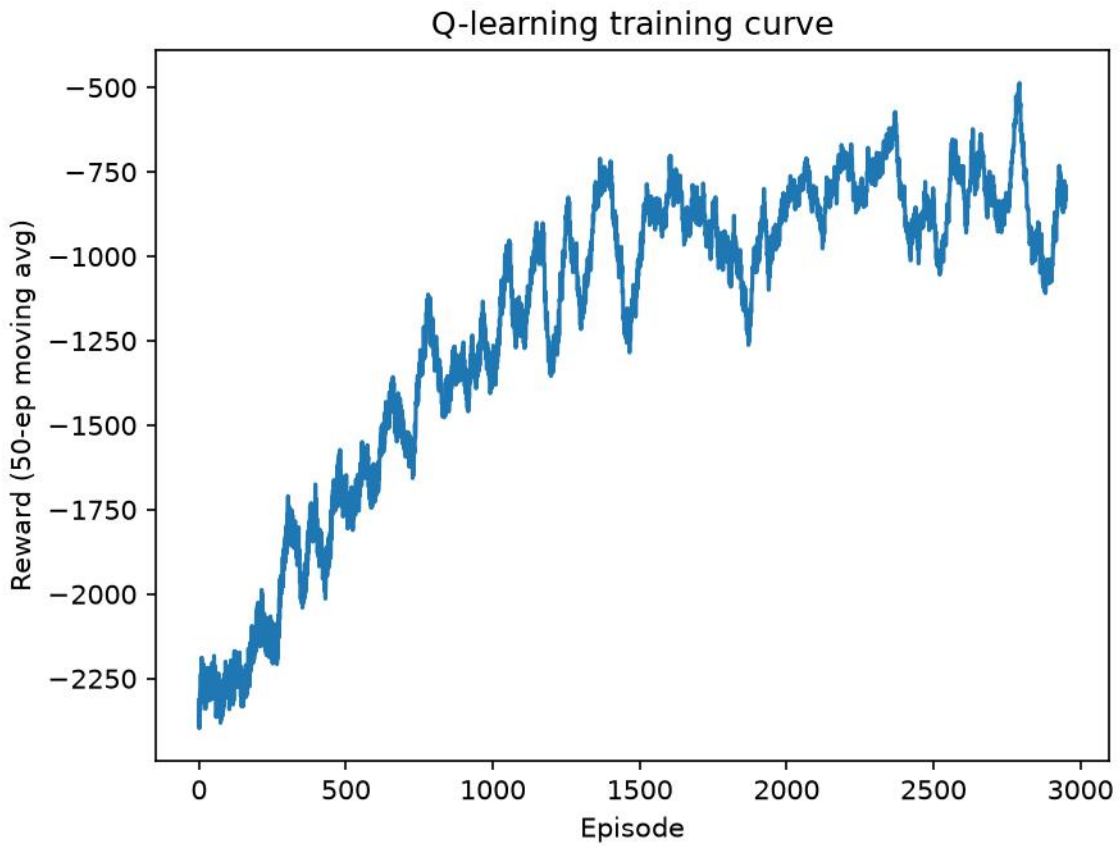
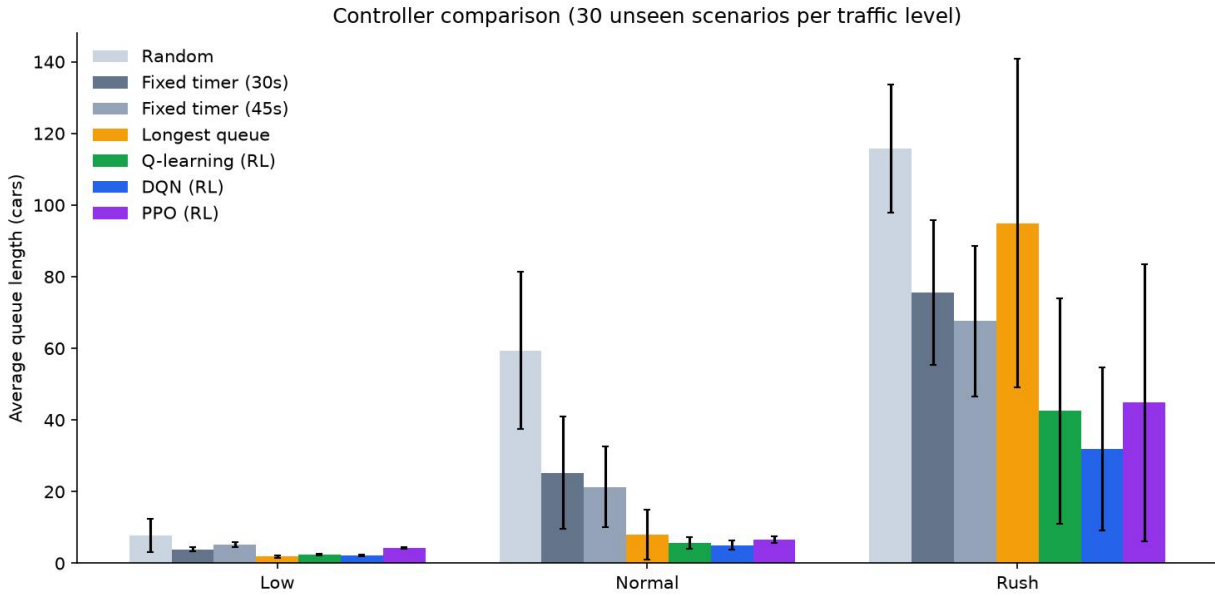
Controller	Low	Normal	Rush
DQN (RL)	2.1 ± 0.2	5.1 ± 1.3	31.9 ± 22.8
PPO (RL)	4.3 ± 0.2	6.6 ± 1.0	44.8 ± 38.6

Table 2. Cars passed in one hour, mean over 30 unseen scenarios. Higher is better.

Controller	Low	Normal	Rush
Random	1108	1764	2121
Fixed timer (30 s)	1114	1912	2423
Fixed timer (45 s)	1111	1929	2493
Longest queue first	1117	1962	2488
Q-learning (RL)	1117	1964	2549
DQN (RL)	1117	1964	2730
PPO (RL)	1115	1962	2704

Table 3. Reduction in average queue by each RL agent (positive means the RL agent is better).

Traffic	RL agent	vs Fixed 30 s	vs Fixed 45 s	vs Longest queue
Low	Q-learning	+37%	+54%	-30%
Low	DQN	+45%	+60%	-13%
Low	PPO	-11%	+18%	-130%
Normal	Q-learning	+78%	+74%	+31%
Normal	DQN	+80%	+76%	+37%
Normal	PPO	+74%	+69%	+18%
Rush	Q-learning	+44%	+37%	+55%
Rush	DQN	+58%	+53%	+66%
Rush	PPO	+41%	+34%	+53%



Discussion and limitations

All three RL agents beat both fixed timers in normal and rush traffic (+69% to +80% lower queue in normal, +34% to +58% in rush), which supports the idea that reacting to queue lengths helps when demand shifts. DQN is best in rush traffic and also passes the most cars there: 2,730 against 2,423 for the 30-second timer, about 13% more.

What the numbers do not show

- In normal traffic Q-learning (5.6 ± 1.6) and DQN (5.1 ± 1.3) are within one standard deviation of each other, so no winner can be named.
- In low traffic the simple longest-queue rule (1.9) is better than every RL agent, and PPO (4.3) is even worse than the 30-second timer (3.9).
- In rush traffic scenarios differ a lot (standard deviations of 20 to 46 cars for the non-random controllers), because arrival rates are close to the intersection's capacity.
- Longest queue first is worse than both fixed timers on average queue in rush traffic (95.0), although it passes more cars than the 30-second timer.
- PPO was the weakest RL agent. It was trained once, for 300,000 steps and with untuned settings, so these results cannot say whether the cause is the algorithm or the training.

Limitations

- One intersection with two phases, no turning lanes and no pedestrians.
- Synthetic random arrivals instead of real traffic data.
- Queues are capped at 60 cars per lane and extra arrivals are dropped, so queue figures understate congestion in rush traffic.
- Each RL agent was trained once, without hyper-parameter tuning.
- No validation in a standard traffic simulator such as SUMO.
- The fixed timers are not tuned for each scenario.

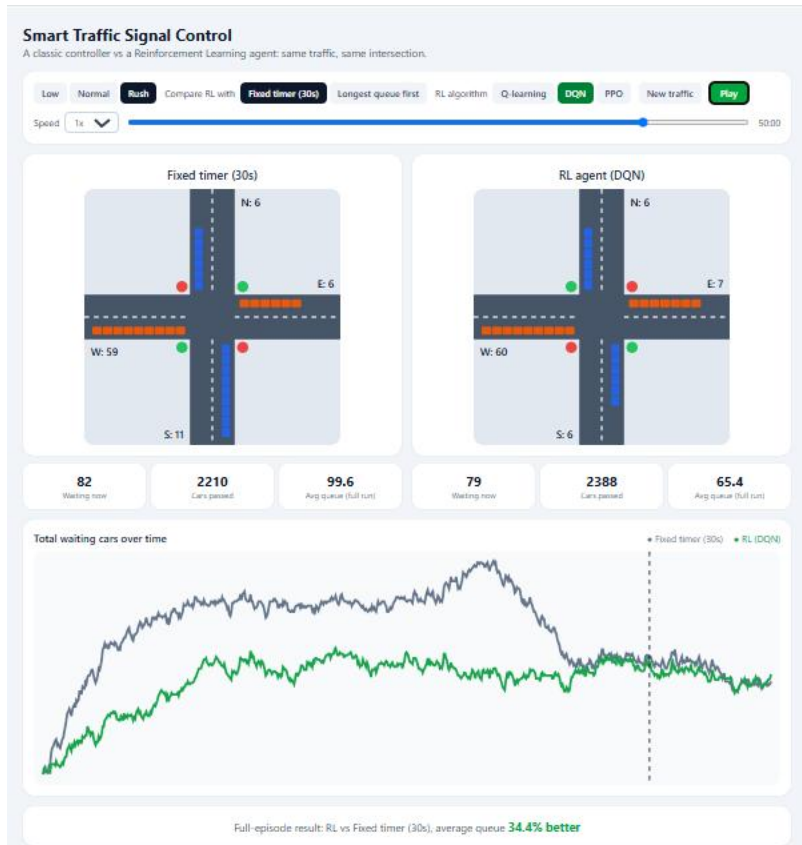
Demo application

The demo runs the same simulator behind a web page, so any baseline can be compared with any trained agent on the same traffic.

- **Backend (Python, FastAPI):** GET /simulate?level=&seed=&controller= runs one simulated hour and returns the queues, phase and cars passed every 5 seconds. GET /controllers lists the controllers whose models are available.
- **Frontend (React, TypeScript, Tailwind CSS):** two intersections drawn side by side with live queues and signal colours; buttons for traffic level (low, normal, rush), baseline (fixed timer or longest queue first) and RL algorithm (Q-learning, DQN, PPO); play, pause, speed and a time slider; and a chart of waiting cars over time.

To run it, start the backend with `python -m uvicorn api:app --port 8000` in the backend folder and the frontend with `npm run dev` in the frontend folder, then open `http://localhost:5173`.

The percentage shown on the page is for one scenario only. The tables in the Results section are averages over 30 scenarios.



Demo screenshot for one rush-traffic scenario (Fixed timer vs DQN). The tables in the Results section report averages over 30 scenarios.

Conclusion and future work

In this simulation, an RL controller reduced waiting at a signalised intersection when traffic shifts: DQN cut the average queue by 80% (normal traffic) and 58% (rush traffic) against a 30-second fixed timer. A simple longest-queue rule remains hard to beat in light traffic, and the gap between Q-learning and DQN in normal traffic is within noise.

Future work, in order of value:

1. Train each agent with several random seeds and report the spread.
2. Test in SUMO with real road layouts and turning movements.
3. Tune DQN and PPO settings and compare with a fixed timer optimised per scenario.
4. Extend to several connected intersections and more than two phases.